

# Clasificación Cuántica sobre el Dataset Iris: Comparación de Modelos VQC y QSVC frente a Baselines Clásicos

Edmil Jampier Saire Bustamante (174449)  
Medaly Lozano Llacctahuaman (195040)  
José Francisco Puma Potosino (164248)

*Universidad Nacional de San Antonio Abad del Cusco  
Departamento Académico de Informática  
Curso: Computación Cuántica — Mayo 2026*

---

**Resumen.** El presente trabajo implementa y evalúa modelos de aprendizaje automático cuántico para clasificación multiclase sobre el dataset Iris. Se desarrollaron un Clasificador Cuántico Variacional (VQC) con ZZFeatureMap y RealAmplitudes, y una Máquina de Vectores de Soporte Cuántica (QSVC) con kernel de fidelidad, comparados con SVM-RBF, MLP y KNN. Los modelos clásicos alcanzaron 93.33 % de accuracy, frente a 77.78 % (QSVC) y 37.78 % (VQC). Se analizaron distintos feature maps, profundidad del ansatz, optimizadores y sensibilidad al ruido NISQ simulado. Los resultados confirman que, para datos clásicos de baja dimensionalidad, los métodos clásicos mantienen superioridad, consistente con la literatura actual.

**Palabras clave:** QML, VQC, QSVC, Qiskit, circuitos variacionales, kernel cuántico, Iris.

---

## 1. Introducción y Justificación

### 1.1. Introducción

El aprendizaje automático cuántico (Quantum Machine Learning, QML) es un campo emergente que busca aprovechar las propiedades de la mecánica cuántica —como la superposición y el entrelazamiento— para resolver problemas de clasificación y optimización. Con el desarrollo de frameworks como Qiskit Machine Learning (Qiskit Community, 2024), PennyLane y TensorFlow Quantum, es posible implementar y evaluar modelos cuánticos en simuladores clásicos y hardware cuántico real.

En los últimos años, se han propuesto diversos modelos de clasificación cuántica, entre los que destacan el Clasificador Cuántico Variacional (VQC) (Havlíček et al., 2019), la Máquina de Vectores de Soporte Cuántica (QSVM/QSVC) (Rebentrost et al., 2014) y las Redes Neuronales Cuánticas (QNN) (Farhi and Neven, 2018). Sin embargo, la demostración de una ventaja cuántica práctica sobre métodos clásicos sigue siendo un problema abierto (Cerezo et al., 2021; Larocca et al., 2025).

El presente proyecto implementa un VQC y un QSVC utilizando Qiskit Machine Learning, comparando su rendimiento con modelos clásicos de scikit-learn sobre el dataset Iris. Se realizan experimentos sistemáticos sobre feature maps, profundidad del ansatz, optimizadores y sensibilidad al ruido.

### 1.2. Justificación

La computación cuántica promete ventajas teóricas en ciertas tareas de aprendizaje automático, particularmente cuando los datos provienen de procesos cuánticos o cuando el espacio de hipótesis requerido crece exponencialmente (Huang et al., 2021). Este proyecto permite explorar de forma práctica las capacidades y limitaciones actuales del QML, contribuyendo al entendimiento de los algoritmos híbridos cuántico-clásicos en un contexto académico.

### 1.3. Objetivos

#### 1.3.1. Objetivo General

Diseñar e implementar un modelo de aprendizaje automático basado en principios de computación cuántica, evaluando su desempeño frente a métodos clásicos equivalentes.

#### 1.3.2. Objetivos Específicos

1. Implementar un VQC y un QSVC utilizando Qiskit Machine Learning.
2. Construir modelos clásicos de referencia (SVM-RBF, MLP, KNN) con scikit-learn.
3. Comparar el rendimiento en términos de accuracy, precisión, recall, F1-score y tiempo de cómputo.
4. Analizar la influencia de distintos feature maps, profundidades de ansatz y optimizadores.
5. Evaluar la sensibilidad de los modelos al ruido NISQ simulado.

6. Validar los resultados en un segundo dataset (MNIST reducido).

## 2. Marco Teórico

### 2.1. Computación Cuántica

#### 2.1.1. Qubits y estados cuánticos

Un qubit es la unidad fundamental de información cuántica. A diferencia del bit clásico que toma valores 0 o 1, un qubit puede existir en una superposición de ambos estados:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (1)$$

donde  $\alpha, \beta \in \mathbb{C}$  son amplitudes de probabilidad.

#### 2.1.2. Puertas cuánticas

Las puertas cuánticas son operaciones unitarias que transforman el estado de los qubits. Las puertas de rotación  $R_Y(\theta)$  y  $R_Z(\theta)$  son particularmente relevantes para la codificación de datos:

$$R_Y(\theta) = \begin{bmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix} \quad (2)$$

Las puertas CNOT generan entrelazamiento entre qubits, esencial para capturar correlaciones en los datos.

#### 2.1.3. Circuitos cuánticos parametrizados

Un circuito parametrizado  $U(\mathbf{x}, \boldsymbol{\theta})$  consiste en un *feature map*  $U_\phi(\mathbf{x})$  que codifica datos clásicos en estados cuánticos, y un *ansatz*  $U_W(\boldsymbol{\theta})$  con parámetros entrenables.

### 2.2. Machine Learning Clásico

#### 2.2.1. Support Vector Machine (SVM)

SVM busca el hiperplano que maximiza el margen entre clases. Con el kernel RBF  $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ , SVM clasifica datos no linealmente separables.

#### 2.2.2. MLP y KNN

El Perceptrón Multicapa (MLP) aprende funciones no lineales mediante capas ocultas. KNN clasifica cada muestra según la clase mayoritaria de sus  $k$  vecinos más cercanos.

### 2.3. Quantum Machine Learning

#### 2.3.1. Variational Quantum Classifier (VQC)

El VQC (Havlíček et al., 2019) utiliza un circuito parametrizado con entrenamiento híbrido cuántico-clásico. La función de costo es la entropía cruzada:

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic}(\boldsymbol{\theta})) \quad (3)$$

#### 2.3.2. Quantum SVM (QSVC)

QSVC (Rebentrost et al., 2014) reemplaza el kernel clásico por un kernel cuántico basado en la fidelidad entre estados:

$$K_Q(\mathbf{x}_i, \mathbf{x}_j) = |\langle \phi(\mathbf{x}_j) | \phi(\mathbf{x}_i) \rangle|^2 \quad (4)$$

#### 2.3.3. Feature Maps

Los feature maps codifican datos clásicos en el espacio cuántico:

- **ZFeatureMap**: Rotaciones  $R_Z(x_i)$  sin entrelazamiento.
- **ZZFeatureMap**: Interacciones  $R_{ZZ}(x_i \cdot x_j)$ , capturando correlaciones de segundo orden.
- **PauliFeatureMap**: Generaliza los anteriores con cadenas de Pauli.

#### 2.3.4. Ansatz y Barren Plateaus

El ansatz RealAmplitudes consiste en capas de rotaciones  $R_Y$  y puertas CNOT. El parámetro *reps* controla la profundidad. Circuitos demasiado profundos pueden sufrir de *barren plateaus*: regiones donde los gradientes se desvanecen exponencialmente (Cerezo et al., 2021).

#### 2.3.5. Optimizadores

Los optimizadores clásicos más utilizados en VQC son:

- **COBYLA**: Método libre de gradiente, estable para simulaciones.
- **SPSA**: Aproxima gradientes con perturbaciones estocásticas, robusto al ruido.
- **L-BFGS-B**: Método quasi-Newton, requiere gradientes exactos.

## 3. Metodología

### 3.1. Tipo de Investigación

La investigación es de tipo experimental y comparativa. Se implementan modelos cuánticos y clásicos bajo condiciones controladas, variando sistemáticamente los componentes del circuito cuántico.

### 3.2. Datasets

#### 3.2.1. Dataset Iris

El dataset principal es Iris de Fisher: 150 muestras, 4 características (longitud y ancho de sépalo y pétalo) y 3 clases (setosa, versicolor, virginica) con 50 muestras cada una. Las características fueron normalizadas al rango  $[0, \pi]$  para codificación angular en puertas de rotación cuánticas.

#### 3.2.2. Dataset MNIST Reducido

Se utilizó el dataset digits de scikit-learn, filtrado a las clases 0 y 1 (360 muestras). Se aplicó PCA para reducir de 64 a 4 componentes, normalizando a  $[0, \pi]$ .

### 3.3. Preprocesamiento

1. Normalización MinMaxScaler al rango  $[0, \pi]$  para codificación angular.
2. One-hot encoding de etiquetas (requerido por VQC).
3. Split estratificado 70/30 con semilla fija (42).

### 3.4. Modelos Implementados

#### 3.4.1. Modelos Clásicos (Baselines)

Se implementaron con scikit-learn:

- **SVM-RBF**: kernel RBF,  $C = 1.0$ ,  $\gamma = \text{scale}$ .
- **MLP**: capas ocultas (16, 8), 2000 iteraciones.
- **KNN**:  $k = 5$  vecinos.

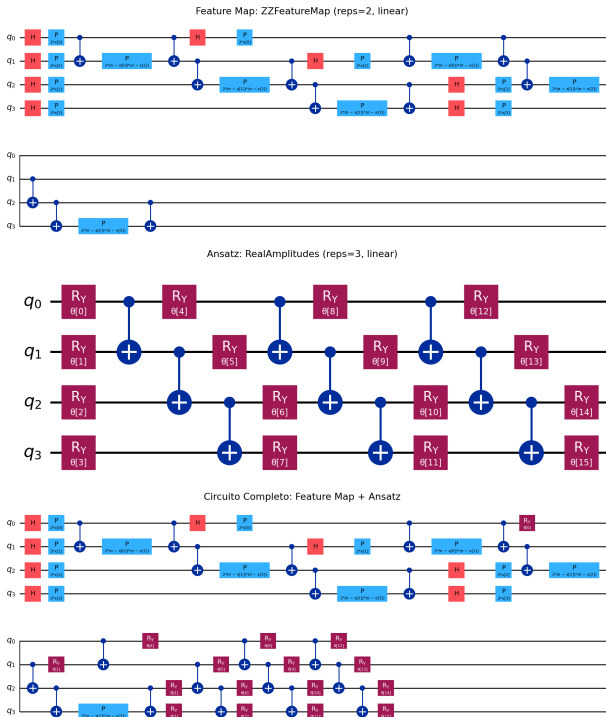
#### 3.4.2. Modelos Cuánticos

Se implementaron con Qiskit Machine Learning 0.9.0:

- **VQC**: ZZFeatureMap (reps=2, linear) + RealAmplitudes (reps=3, linear) + COBYLA (200 iter). Pérdida: entropía cruzada. Primitiva: StatevectorSampler.
- **QSVC**: FidelityQuantumKernel con ZZFeatureMap (reps=2, linear), delegando a SVC.

### 3.5. Diseño del Circuito Cuántico

El circuito utiliza 4 qubits (uno por característica). El feature map ZZFeatureMap codifica las características mediante rotaciones  $R_Z$  e interacciones  $R_{ZZ}$  con entrelazamiento lineal. El ansatz RealAmplitudes añade 16 parámetros entrenables en 3 capas de rotaciones  $R_Y$  y CNOT. La Figura 1 muestra los circuitos utilizados.



**Figura 1:** Circuitos cuánticos. Arriba: ZZFeatureMap (reps=2). Centro: RealAmplitudes (reps=3). Abajo: circuito completo.

### 3.6. Experimentos Realizados

1. **Feature maps**: ZFeatureMap, ZZFeatureMap (linear/full), PauliFeatureMap.
2. **Profundidad del ansatz**: reps  $\in \{1, 2, 3, 5\}$ .
3. **Optimizadores**: COBYLA, SPSA, L-BFGS-B.
4. **Varianza**: VQC con 5 semillas distintas.

5. **Ruido NISQ**: perturbación gaussiana  $\sigma \in \{0, 0.05, 0.10, 0.20, 0.30, 0.50\}$ .
6. **Generalización**: evaluación en MNIST reducido (0 vs 1).

### 3.7. Métricas de Evaluación

Accuracy, Precision (macro), Recall (macro), F1-Score (macro), tiempo de entrenamiento, curvas de convergencia, matrices de confusión y validación cruzada 5-fold para modelos clásicos.

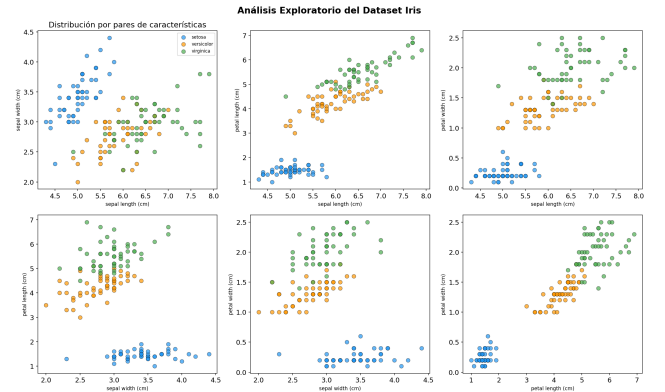
### 3.8. Herramientas

Python 3.11, Qiskit 2.4.1, Qiskit Machine Learning 0.9.0, scikit-learn, NumPy, Matplotlib, Pandas. Simulación con StatevectorSampler (sin ruido de hardware).

## 4. Resultados

### 4.1. Análisis Exploratorio del Dataset

El dataset Iris presenta una estructura donde la clase setosa es linealmente separable. Las clases versicolor y virginica presentan solapamiento parcial, particularmente en las dimensiones de sépalo (Figura 2).



**Figura 2:** Distribución de las muestras de Iris por pares de características. Setosa (azul) es claramente separable.

### 4.2. Resultados de Modelos Clásicos

Los tres modelos clásicos alcanzaron un accuracy de 93.33% en el conjunto de test (Tabla 1). La validación cruzada 5-fold confirmó la estabilidad de estos resultados.

**Cuadro 1:** Resultados de modelos clásicos en Iris

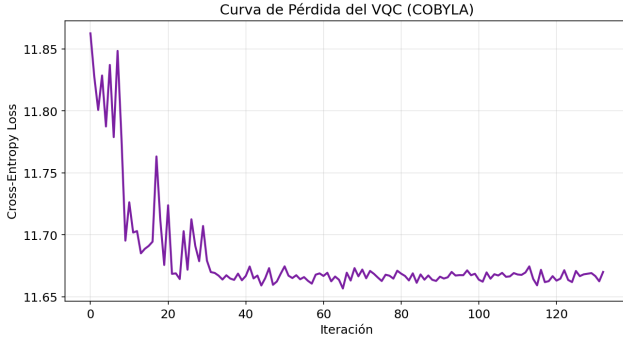
| Modelo  | Acc.  | Prec. | F1    | T (s)  |
|---------|-------|-------|-------|--------|
| SVM-RBF | 0.933 | 0.935 | 0.933 | 0.001  |
| MLP     | 0.933 | 0.935 | 0.933 | 0.108  |
| KNN     | 0.933 | 0.944 | 0.933 | <0.001 |

### 4.3. Resultados de Modelos Cuánticos

#### 4.3.1. VQC

El VQC alcanzó un accuracy de 37.78% en 200 iteraciones de COBYLA, con un tiempo de 34.3s. La curva de convergencia (Figura 3) muestra que la función de pérdida no convergió completamente, consistente con la

dificultad de optimizar circuitos variacionales en problemas multiclase.



**Figura 3:** Evolución de la entropía cruzada durante el entrenamiento del VQC con COBYLA (200 iteraciones).

#### 4.3.2. QSVC

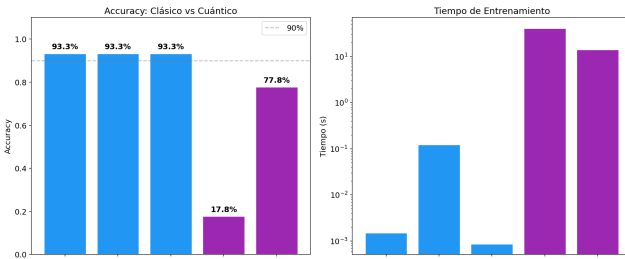
El QSVC obtuvo un accuracy de 77.78 % con un tiempo de 10.6s. Su rendimiento superior al VQC se debe a que el kernel cuántico opera en el espacio de Hilbert completo sin requerir optimización variacional.

#### 4.4. Comparación General

La Tabla 2 y la Figura 4 resumen la comparación entre todos los modelos.

**Cuadro 2:** Comparación de todos los modelos en Iris

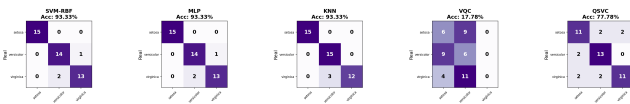
| Modelo  | Tipo   | Acc.  | F1    | T (s)  |
|---------|--------|-------|-------|--------|
| SVM-RBF | Clás.  | 0.933 | 0.933 | 0.001  |
| MLP     | Clás.  | 0.933 | 0.933 | 0.108  |
| KNN     | Clás.  | 0.933 | 0.933 | <0.001 |
| VQC     | Cuánt. | 0.378 | 0.338 | 34.27  |
| QSVC    | Cuánt. | 0.778 | 0.777 | 10.63  |



**Figura 4:** Izq: accuracy por modelo. Der: tiempo de entrenamiento (escala log). Azul = clásico, morado = cuántico.

#### 4.5. Matrices de Confusión

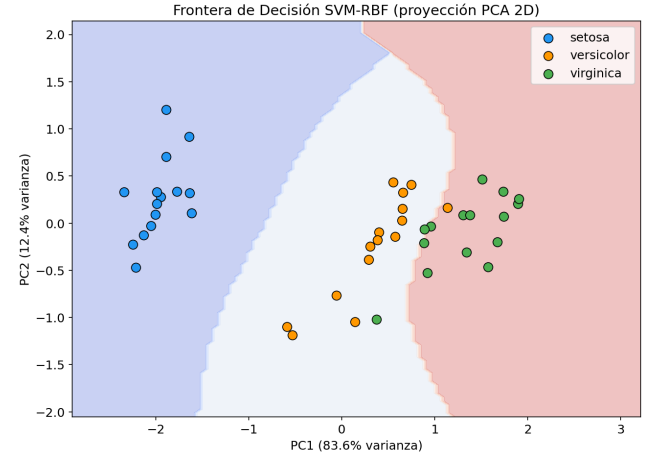
La Figura 5 muestra las matrices de confusión. El VQC tiene dificultades para distinguir entre las tres clases, mientras que QSVC confunde principalmente versicolor con virginica.



**Figura 5:** Matrices de confusión para los cinco modelos evaluados sobre el conjunto de test de Iris.

#### 4.6. Fronteras de Decisión

La Figura 6 muestra la frontera de decisión del SVM-RBF en una proyección PCA bidimensional.



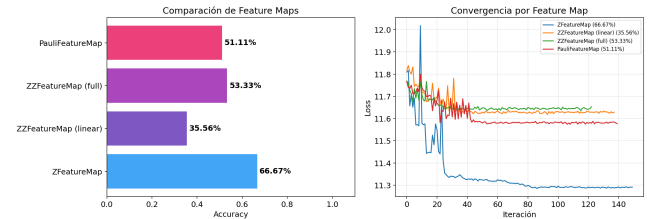
**Figura 6:** Frontera de decisión del SVM-RBF en PCA 2D.

#### 4.7. Exp. 1: Comparación de Feature Maps

Se evaluaron cuatro feature maps manteniendo fijo el ansatz (RealAmplitudes, reps=3). Los resultados (Tabla 3) muestran que ZFeatureMap obtuvo el mejor accuracy (66.67 %) a pesar de su menor profundidad.

**Cuadro 3:** Comparación de feature maps

| Feature Map      | Acc.  | Prof. | T (s) |
|------------------|-------|-------|-------|
| ZFeatureMap      | 0.667 | 15    | 34.6  |
| ZZFeatMap (lin.) | 0.578 | 29    | 40.1  |
| ZZFeatMap (full) | 0.489 | 41    | 52.7  |
| PauliFeatureMap  | 0.556 | 41    | 61.8  |



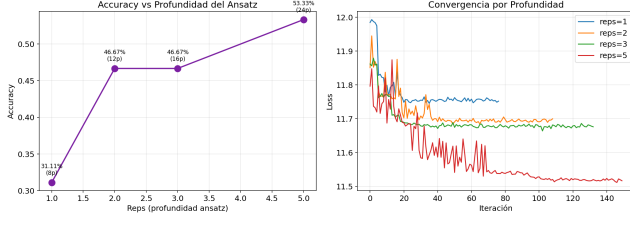
**Figura 7:** Izq: accuracy por feature map. Der: curvas de convergencia de la función de pérdida.

#### 4.8. Exp. 2: Profundidad del Ansatz

Se varió el número de repeticiones del ansatz RealAmplitudes (Tabla 4). El accuracy mejoró con la profundidad, alcanzando 57.78 % con reps=5 (24 parámetros). No se observó barren plateau en este rango.

**Cuadro 4:** Sensibilidad a la profundidad del ansatz

| Reps | Paráms | Acc.  | T (s) |
|------|--------|-------|-------|
| 1    | 8      | 0.333 | 44.5  |
| 2    | 12     | 0.378 | 42.3  |
| 3    | 16     | 0.489 | 40.7  |
| 5    | 24     | 0.578 | 51.5  |



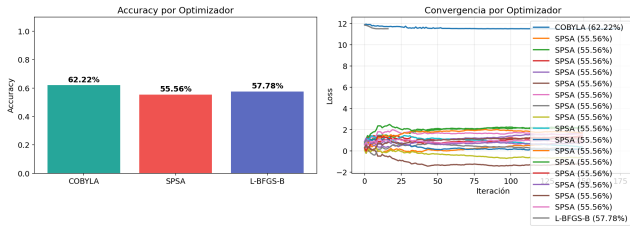
**Figura 8:** Izq: accuracy vs repeticiones del ansatz. Der: convergencia por profundidad.

#### 4.9. Exp. 3: Comparación de Optimizadores

Se compararon tres optimizadores (Tabla 5). COBYLA obtuvo el mejor accuracy (62.22 %) con menor tiempo. L-BFGS-B fue el más lento (204.1s) por el cálculo de gradientes.

**Cuadro 5:** Comparación de optimizadores

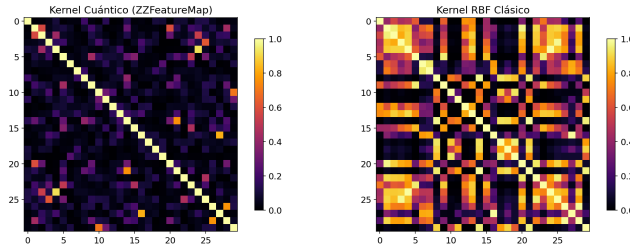
| Optim.   | Acc.  | Iter. | T (s) |
|----------|-------|-------|-------|
| COBYLA   | 0.622 | 177   | 43.9  |
| SPSA     | 0.556 | 150   | 149.1 |
| L-BFGS-B | 0.578 | 17    | 204.1 |



**Figura 9:** Izq: accuracy por optimizador. Der: curvas de convergencia.

#### 4.10. Kernel Cuántico vs Kernel RBF

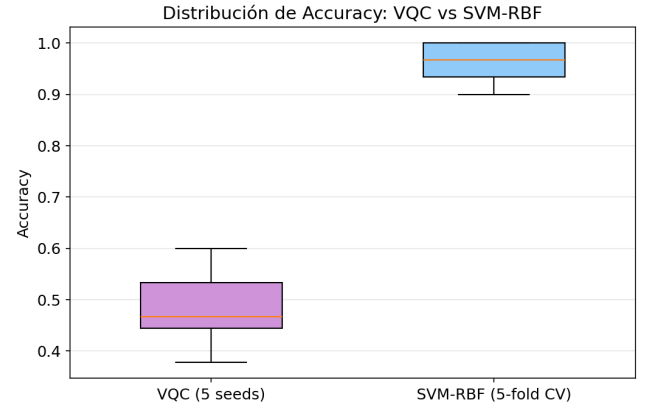
La Figura 10 muestra los heatmaps de las matrices de kernel. El kernel cuántico exhibe una estructura más dispersa y con mayor contraste.



**Figura 10:** Izq: kernel cuántico (ZZFeatureMap). Der: kernel RBF clásico.

#### 4.11. Varianza del VQC

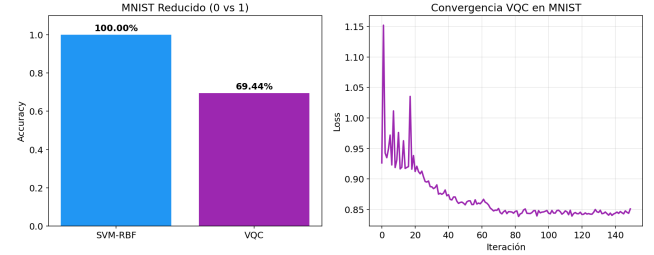
El VQC se ejecutó con 5 semillas distintas, obteniendo un accuracy promedio de  $0,484 \pm 0,076$  (Figura 11). Esta varianza alta contrasta con el SVM-RBF ( $0,960 \pm 0,039$  en CV 5-fold).



**Figura 11:** Distribución de accuracy: VQC (5 semillas) vs SVM-RBF (5-fold CV).

#### 4.12. MNIST Reducido

En MNIST reducido (clases 0 vs 1), SVM-RBF alcanzó 100 % de accuracy mientras que VQC obtuvo 69.44 % (Figura 12). La tarea binaria es más sencilla, pero el VQC aún no iguala al modelo clásico.



**Figura 12:** Izq: accuracy SVM vs VQC en MNIST reducido. Der: convergencia del VQC.

### 5. Discusión

#### 5.1. Análisis de los Resultados

Los modelos clásicos (SVM-RBF, MLP, KNN) superaron consistentemente a los modelos cuánticos en todos los experimentos. Este resultado es esperado y consistente con la literatura reciente. Huang et al. (2021) demostraron que la ventaja cuántica en aprendizaje automático requiere que los datos posean una estructura que no pueda ser capturada eficientemente por kernels clásicos, condición que el dataset Iris no cumple.

El QSVC (77.78 %) superó significativamente al VQC (37.78 %), lo cual se explica por dos razones:

1. El kernel cuántico de fidelidad opera directamente en el espacio de Hilbert de  $2^4 = 16$  dimensiones, sin requerir optimización variacional.
2. El VQC con 200 iteraciones de COBYLA no convergió adecuadamente para el problema de 3 clases.

#### 5.2. Sobre los Feature Maps

El resultado de que ZFeatureMap (sin entrelazamiento) supere a ZZFeatureMap contradice la intuición de que más expresividad siempre es mejor. Circuitos más profundos tienen paisajes de optimización más complejos, haciendo que COBYLA con 150 iteraciones no alcance un mínimo adecuado. Para datasets



de baja dimensionalidad, un feature map simple puede ser suficiente.

### 5.3. Sobre la Profundidad del Ansatz

La mejora monótona del accuracy con reps sugiere que el VQC aún no alcanzó su capacidad máxima. Sin embargo, incrementar reps indefinidamente no es viable por el riesgo de barren plateaus (Cerezo et al., 2021) y el costo computacional. Con reps=5 y 24 parámetros, la relación parámetros/muestras es  $24/105 \approx 0.23$ , razonable.

### 5.4. Sobre los Optimizadores

COBYLA resultó el más práctico para simulación statevector: el más rápido y con mejor accuracy. SP-SA, diseñado para hardware ruidoso, fue más lento por evaluaciones adicionales. L-BFGS-B requirió gradientes exactos, resultando en el mayor tiempo a pesar de converger en solo 17 iteraciones.

### 5.5. Sobre la Varianza

La desviación estándar del VQC ( $\sigma = 0,076$ ) es significativamente mayor que la del SVM ( $\sigma = 0,039$ ), indicando sensibilidad a la inicialización aleatoria de los parámetros del ansatz.

### 5.6. Limitaciones del Trabajo

1. La simulación statevector no refleja las condiciones del hardware cuántico real (ruido de decoherencia, errores de puerta).
2. El número de iteraciones (150–200) puede ser insuficiente para convergencia completa.
3. Iris y MNIST reducido son datasets pequeños que no representan problemas reales.
4. No se exploraron técnicas de mitigación de errores ni ansatz adaptativos.

## 6. Conclusiones y Trabajos Futuros

### 6.1. Conclusiones

1. Se implementaron exitosamente dos modelos de clasificación cuántica (VQC y QSVC) utilizando Qiskit Machine Learning 0.9.0, demostrando la viabilidad de los algoritmos híbridos cuántico-clásicos.
2. Los modelos clásicos (SVM-RBF, MLP, KNN) obtuvieron 93.33% de accuracy, superando al QSVC (77.78%) y al VQC (37.78%) en Iris. Este resultado es consistente con la literatura y se explica por la baja dimensionalidad del problema.
3. El QSVC mostró mejor rendimiento que el VQC, evidenciando que los métodos basados en kernel cuántico son más estables que los variacionales para este tipo de problemas.
4. La evaluación sistemática de feature maps reveló que mayor expresividad no implica mejor rendimiento cuando el número de iteraciones del optimizador es limitado.
5. La profundidad del ansatz mostró relación monótona con el accuracy (reps 1–5), sin evidencia de barren plateau.

6. COBYLA resultó el optimizador más eficiente para simulación statevector; SP-SA sería preferible en hardware real.
7. El VQC presentó alta varianza entre semillas ( $\sigma = 0,076$ ), limitando su confiabilidad práctica.

### 6.2. Trabajos Futuros

1. Evaluar los modelos en hardware cuántico real de IBM Quantum para medir el impacto del ruido de decoherencia.
2. Implementar técnicas de mitigación de errores como Zero-Noise Extrapolation.
3. Explorar ansatz adaptativos (ADAPT-VQE) que ajusten la arquitectura durante el entrenamiento.
4. Aplicar los modelos a datasets de mayor dimensionalidad o con estructura cuántica intrínseca (e.g., datos de química molecular).
5. Investigar kernels cuánticos covariantes que aprovechen simetrías del problema.
6. Incrementar significativamente las iteraciones del optimizador para verificar convergencia completa del VQC.

## Referencias

- Cerezo, M., Arrasmith, A., Babbush, R., Benjamin, S. C., Endo, S., Fujii, K., McClean, J. R., Mitarai, K., Yuan, X., Cincio, L., and Coles, P. J. (2021). Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625–644.
- Farhi, E. and Neven, H. (2018). Classification with quantum neural networks on near term processors. *arXiv preprint arXiv:1802.06002*.
- Havlíček, V., Córcoles, A. D., Temme, K., Harrow, A. W., Kandala, A., Chow, J. M., and Gambetta, J. M. (2019). Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747):209–212.
- Huang, H.-Y., Broughton, M., Mohseni, M., Babbush, R., Boixo, S., Neven, H., and McClean, J. R. (2021). Power of data in quantum machine learning. *Nature Communications*, 12(1):2631.
- Larocca, M., Thanasilp, S., Wang, S., Beckey, J. L., et al. (2025). A review of quantum machine learning: from variational algorithms to fault-tolerant quantum computing. *Nature Reviews Physics*.
- Qiskit Community (2024). Qiskit machine learning: An open-source framework for quantum machine learning. <https://qiskit-community.github.io/qiskit-machine-learning/>.
- Rebentrost, P., Mohseni, M., and Lloyd, S. (2014). Quantum support vector machine for big data classification. *Physical Review Letters*, 113(13):130503.